

Reviewer Pack — Minimal Hodge Index of Curves in Characteristic p Bounds Res...

Entry 402a394e8211 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-30 00:48:07 UTC

Minimal Hodge Index of Curves in Characteristic p Bounds Resolution Proof Size for k -CNF

Entry ID: 402a394e8211 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-30 00:48:07 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Curves over finite fields) **Field B** (complexity object): Boolean Function Complexity: Resolution Proof Size

Statement:

For every k -CNF F with n variables, the minimal Hodge index of a curve in characteristic p corresponding to F is $\Theta(\log n)$, where the resolution proof size $t(F)$ is $\Omega(\log n)$. Equivalently, for all k -CNFs F , if there exists a curve C in characteristic p such that the minimal Hodge index $h(C) \leq k/p^n$, then $t(F) = O(h(C))$.

Rationale (proposer's reasoning):

Algebraic geometry over finite fields has not been extensively applied to complexity theory, particularly resolution proof size. The conjecture leverages the geometric invariant of Hodge index, which is computable for curves over finite fields, and suggests a potential connection to circuit lower bounds. If true, it would provide a novel approach to proving lower bounds on resolution proof size.

Taxonomy category: cg_kw_andreev (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): b2e04acfbe699565

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the correlation coefficient between the minimal Hodge index $h(C)$ and $\log_2(n)$ is statistically significant ($p\text{-value} \leq 0.05$) across 30 random seeds.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "minimal Hodge index" AND "curves over finite fields" AND resolution proof size" - "characteristic p" AND curve AND "Boolean function complexity" - "Hodge index" AND characteristic p AND CNF formula resolution

Top relevant hits considered: - [http://arxiv.org/abs/2511.08780v1] Around a class version of the Hodge index theorem for singular varieties - [http://arxiv.org/abs/1001.4788v1] Positivity of heights of codimension 2 cycles over function field of characteristic 0 - [http://arxiv.org/abs/2601.07933v1] p-Curvature and Non-Abelian Cohomology

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=0, elapsed=0.0s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def is_prime(n):
        if n <= 1:
            return False
        for i in range(2, int(math.sqrt(n)) + 1):
            if n % i == 0:
                return False
        return True

    def construct_curve(n):
        # Placeholder for curve construction logic
        # This is a dummy implementation and should be replaced with actual logic
        return random.randint(1, n)

    def hodge_index(h):
        # Placeholder for Hodge index calculation
        # This is a dummy implementation and should be replaced with actual logic
        return math.log2(h) if h > 0 else -math.inf

    def resolution_proof_size(n):
        # Placeholder for resolution proof size calculation
        # This is a dummy implementation and should be replaced with actual logic
        return math.log2(n) if n > 1 else -math.inf
```

```

n_values = [10, 20, 40]
hodge_indices = []
proof_sizes = []

for n in n_values:
    if not is_prime(n):
        continue
    h = construct_curve(n)
    hodge_indices.append(hodge_index(h))
    proof_sizes.append(resolution_proof_size(n))

if len(hodge_indices) < 30:
    return {
        "metric_name": "correlation_coefficient",
        "metric_value": None,
        "instances_tested": len(hodge_indices),
        "n_max": max(n_values),
        "conjecture_holds": False,
        "counterexample": "not_enough_instances"
    }

correlation = 0
for h, p in zip(hodge_indices, proof_sizes):
    correlation += (h - sum(hodge_indices) / len(hodge_indices)) * (p - sum(proof_sizes) / len(proof_sizes))
correlation /= math.sqrt(sum((h - sum(hodge_indices) / len(hodge_indices)) ** 2 for h in hodge_indices) + sum((p - sum(proof_sizes) / len(proof_sizes)) ** 2 for p in proof_sizes))

return {
    "metric_name": "correlation_coefficient",
    "metric_value": correlation,
    "instances_tested": len(hodge_indices),
    "n_max": max(n_values),
    "conjecture_holds": abs(correlation) > 0.95,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results if r["metric_value"] is not None) / len(results)
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value) ** 2 for r in results if r["metric_value"] is not None) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(s for s, r in zip(seeds, results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"not_supported\" first_failing_seed={first_failing_seed}")

```

```

else:
    print("RESULT: INCONCLUSIVE no_data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

me': 'correlation_coefficient', 'metric_value': None, 'instances_tested': 0, 'n_max': 40, 'conjecture':
TRIAL: {'metric_name': 'correlation_coefficient', 'metric_value': None, 'instances_tested': 0, 'n_max': 40, 'conjecture':
TRIAL: {'metric_name': 'correlation_coefficient', 'metric_value': None, 'instances_tested': 0, 'n_max': 40, 'conjecture':
TRIAL: {'metric_name': 'correlation_coefficient', 'metric_value': None, 'instances_tested': 0, 'n_max': 40, 'conjecture':
TRIAL: {'metric_name': 'correlation_coefficient', 'metric_value': None, 'instances_tested': 0, 'n_max': 40, 'conjecture':
TRIAL: {'metric_name': 'correlation_coefficient', 'metric_value': None, 'instances_tested': 0, 'n_max': 40, 'conjecture':
TRIAL: {'metric_name': 'correlation_coefficient', 'metric_value': None, 'instances_tested': 0, 'n_max': 40, 'conjecture':
TRIAL: {'metric_name': 'correlation_coefficient', 'metric_value': None, 'instances_tested': 0, 'n_max': 40, 'conjecture':
TRIAL: {'metric_name': 'correlation_coefficient', 'metric_value': None, 'instances_tested': 0, 'n_max': 40, 'conjecture':
TRIAL: {'metric_name': 'correlation_coefficient', 'metric_value': None, 'instances_tested': 0, 'n_max': 40, 'conjecture':
TRIAL: {'metric_name': 'correlation_coefficient', 'metric_value': None, 'instances_tested': 0, 'n_max': 40, 'conjecture':
RESULT: FALSIFIED counterexample="not_supported" first_failing_seed=11

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

The test code uses dummy implementations for curve construction, Hodge index calculation, and resolution proof size calculation, which do not correspond to the actual mathematical concepts described in the conjecture. The test is therefore not measuring what it claims to measure.

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Safety rail: critic_challenge_falsified | original: The test code uses dummy implementations for curve construction, Hodge index calculation, and resolution proof size calculation, which do not correspond to the actual mathematical concepts described in the conjecture. | next: Develop a proper test code that accurately implements curve construction, Hodge index calculation, and resolution proof size calculation according to the mathematical concepts described in the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	27641
2	preregistration	ollama_remote	glm4:latest	0	0	21832
3	novelty	ollama_remote	glm4:latest	0	0	17223
4	novelty	ollama_remote	glm4:latest	0	0	22235
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17583
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9000
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7278

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10194
9	critic	ollama_remote	glm4:latest	0	0	13879
10	judge	ollama_remote	glm4:latest	0	0	16765

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 163631 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in research/audit/402a394e8211.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/402a394e8211.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/402a394e8211.tar.gz (if generated)